Daniil Grigoriev

Systems Programmer · Compiler Engineer

daniilextra13@gmail.com daniilgriga

+7 (911) 909 61 83 @dangrigoriev

EDUCATION

Moscow Institute of Physics and Technology (MIPT) 2024 – Present
Bachelor of Applied Mathematics and Physics Dolgoprudny, Russia
Dept. of Radio-Engineering and Computer Science **GPA: 7.21 / 10**

Relevant Courses (semester / grade)

System Programming & Compiler Technology — I. Dedinskiy, MIPT	1	**8 / 10**
Intro to Computer Architecture and Assembler — I. Dedinskiy, MIPT	2	**7 / 10**
Algorithms & Data Structures — VK Education	3	**10 / 10**
Industrial C++ Programming — K. Vladimirov, MIPT	3–4	**9 / 10 ***
SQL & Databases / Business Programming Paradigms — MIPT	3–4	**8 / 10 ***
Intro to Computer Systems Architecture — MIPT	4	**in progress**
Intro to Tensor Compilers — MIPT	4	**in progress**

* ongoing — grade for completed semester

PROJECTS

ParaCL [Team Project] [In Progress] 2026
C-like language compiler & interpreter pipeline: lexer → parser → AST → tree-traversal interpreter *and* LLVM IR backend for native code generation. Supports variables, control flow, and I/O.
C++ LLVM Flex/Bison GoogleTest CMake

Tensor Compiler [Team Project] [In Progress] 2026
Frontend stage implemented — ONNX model ingestion, conversion to an internal compute graph IR, graph consistency verification, and Graphviz export for visualization. Optimizations and code-generation backend are under development.
C++ ONNX Protobuf GoogleTest CMake

AngryMIPTs [Team Project] [In Progress] 2026
Physics-based projectile game (Angry Birds-inspired). Multithreaded architecture with physics and rendering fully decoupled via lock-free inter-thread communication. Cross-platform: native desktop and browser (in debug) builds through a unified platform abstraction layer. Online leaderboard with JWT-authenticated score submission; 9 projectile types each with a distinct special ability.
C++ Box2D SFML CMake GoogleTest nlohmann_json PostgreSQL

Bitonic Sort — OpenCL [Team Project] [In Progress] 2026

GPU-accelerated bitonic sort with OpenCL kernels. Includes a CPU baseline, benchmarking across input sizes, and analysis of host → device data-transfer overhead.

C++ OpenCL GoogleTest CMake

Custom (Cook) Programming Language May 2025

Turing-complete esoteric language (*Cook*) with a self-contained compiler that emits native x86-64 ELF64 executables directly — no assembler or linker required. Full three-stage pipeline: frontend (lexer + recursive-descent parser → AST) → middle-end (constant folding, algebraic simplifications) → backend (IR → x86-64 machine code → ELF64 binary with two `PT_LOAD` segments, virtual registers mapped to `rax`–`r15`).

C x86-64 ELF64 Graphviz make

Hash Table — SIMD Optimization May 2025

Lab study of hash table performance: compared 10 hash functions by distribution variance, then applied three optimization passes — AVX2 `strcmp` via YMM registers, CRC32 in NASM (SSE4.2), and inline assembly CRC32. Profiled with `perf` and `callgrind` at each stage.

C x86-64 NASM AVX2/SSE4.2 make perf

Mandelbrot Set — SIMD Visualization April 2025

Real-time interactive visualization with three implementations compared: scalar, compiler-auto-vectorized (arrays of 8 floats), and hand-written AVX intrinsics. Benchmarked across compilers.

C/C++ AVX2 SFML make

SKILLS

Languages	C, C++ (proficient); x86-64 Assembly (experienced); Python, PostgreSQL (familiar)
Compilers	LLVM IR, Flex/Bison, SLR(1), custom lexer/parser/IR pipelines, ELF64 code-gen
Performance	SIMD (AVX/AVX2/SSE), OpenCL, multithreading, profiling (`perf`, `callgrind`), benchmarking
Tooling	Git, GDB, Valgrind, ASan/UBSan, Ghidra, CMake, Make, GoogleTest, Godbolt
Spoken	Russian (native), English (intermediate)